

BICOL UNIVERSITY

College of Science, Department of Information Technology
Learning Module, Student Copy

Course Code & Title	IT 124 – Capstone Project 2
Module Title	Learning Module 01: System Design Refinement
Group Name	
Year - Block	
Estimated Time	60 to 90 minutes for the concept review and learning activities. Your instructor will time the Final Assessment separately.
Materials Needed	Your team's approved Capstone 1 proposal and panel recommendation sheet; a laptop or mobile device with internet access; access to a diagramming tool, a database or ERD modeling tool, and a UI/UX design tool (proprietary or open-source options are both acceptable); a pen or word processor for written answers.

What You Will Be Able to Do

This module continues directly from your Capstone 1 proposal defense. Your panel already reviewed your proposed system and gave your team specific recommendations. This module walks you through the process of turning those recommendations into a finalized, build-ready design, and it gives you practice applying that process before your instructor checks your work.

1	By the end of this module, you will be able to finalize your system's architecture, database schema, and process/data flow diagrams, based on the recommendations your panel gave you at your Capstone 1 proposal defense.
2	You will be able to refine the user interface (UI) and user experience (UX) design of your proposed system.
3	You will be able to justify your design decisions in terms of functionality, security, scalability, and alignment with the problem your project identified.

PERFORMANCE STANDARD

By the end of this module, your team should be able to produce a complete and validated System Design Document. That document must include your finalized architecture, your database schema (ERD), and your UI/UX mockups, and it must show, item by item, how you incorporated your panel's recommendations from Capstone Project 1.

How to Use This Module

- **Read the concept review first.** It is a condensed refresher of what your instructor covered in the lecture discussion. It is not a replacement for that discussion.
- **Study the worked model carefully.** It shows one capstone team's design refinement process from start to finish. You will follow the same process for your own project.
- **Complete the learning activities in order.** Each tier builds on the one before it: first you check your understanding of the terms, then you apply the concepts to a scenario and to your own project, and finally you analyze and justify design decisions.
- **Attempt the Final Assessment only after finishing the activities.** It covers the same material at a deeper level, and your instructor will check it against your activity answers for consistency.
- **Write directly on this module.** Use the ruled lines and tables provided. If you need more space, continue on a separate sheet and label it clearly with the item number.

Concept Review

This is a condensed review of the design refinement process. For the full discussion, refer to your notes from the lecture session.

1. Finalizing Your System Architecture

Your Capstone 1 panel likely flagged one of three things about your architecture: a component you left unspecified, a technology choice they questioned as unsuitable for your system's scale, or an integration point (such as a payment gateway or an SMS gateway) that still needs a defined interface. Finalizing your architecture means resolving each of those flags directly, and documenting how you resolved them.

Most capstone systems use one of four patterns. A layered (n-tier) architecture separates presentation, business logic, and data into distinct layers, and is the most common capstone choice because it is easy for a small team to build and test. A client-server architecture describes the general split between your client application and your server. Model-View-Controller (MVC) and related patterns structure the application layer itself into model, view, and controller, which most frameworks (Laravel, Django, .NET MVC) already assume. A modular monolith, a single deployable application with clearly separated internal modules, is usually the right middle ground when a system has several loosely related functional areas; true microservices are rarely necessary at capstone scale.

Your finalized architecture documentation needs an architecture diagram, a short written justification for why you chose this pattern over the alternatives, a one-sentence responsibility statement for every component, and a technology stack table.

2. Refining Your Database Schema (ERD)

A proposal-stage data model almost always has gaps: entities implied by your narrative but never drawn, attributes with no defined data type, relationships whose cardinality was assumed rather than checked, and missing audit fields such as timestamps or status flags. Refining your schema means closing every one of those gaps. Run every entity and relationship through six checks: entity completeness, attribute definition (including marking password fields as hashed, never plain text), primary and foreign keys, cardinality and participation, normalization to at least Third Normal Form (3NF), and referential integrity and cascade rules for every foreign key.

Use Crow's Foot notation for your ERD, since it expresses cardinality directly on the connecting line between two entities. Your finished schema deliverable is the ERD plus a data dictionary listing every entity, attribute, type, constraint, and short description.

3. Process and Data Flow Diagrams

A Data Flow Diagram (DFD) shows how data moves between external entities, processes, and data stores. A Context Diagram (Level 0) shows your entire system as one process with only its boundary and external entities. A Level 1 DFD decomposes that process into major subprocesses and shows the data stores each one uses. Deeper levels are only needed where a subprocess is complex enough to warrant it. For algorithm-heavy logic that a DFD

cannot express clearly, use a flowchart or a UML sequence diagram instead; a sequence diagram is especially useful for showing how your client, API layer, and database interact for one transaction.

Before finalizing your diagrams, check that every data store in your DFD maps to a specific entity in your ERD, and that every process maps to a component in your architecture diagram. This cross-check is exactly what your Design Review Checklist looks for.

4. Refining Your UI/UX Design

Refinement means moving from a rough wireframe to a validated, high-fidelity mockup: pixel-accurate layouts, a consistent color and typography system, and every screen a user will actually encounter, including error states, empty states, and confirmation dialogs. Check your interface against Nielsen's usability heuristics, particularly visibility of system status, match between the system and the real world, user control and freedom, consistency and standards, error prevention and recovery, and accessibility. Where possible, walk your mockups through with a real representative of your intended end users before finalizing them, and document any change that results.

5. Justifying Your Design Decisions

A design is complete only when your team can defend it, not just when it works. Every major decision needs a written justification along four dimensions. Functionality: does the design fully support the functional requirements from your Capstone 1 proposal, traced by requirement number? Security: name the specific mechanism, not just the goal (for example, state that passwords are hashed with bcrypt, rather than simply claiming the system is secure). Scalability: state the expected scale the design was built for, and the concrete choices, such as indexed keys or pagination, that support it. Problem alignment: does the design still solve the problem your project originally identified? This four-part structure, functionality, security, scalability, and problem alignment, is exactly what you will be asked to produce in this module's activities and Final Assessment.

Guided Practice: A Worked Model

Read the following example carefully before starting the activities. It shows the complete process, from panel recommendation to finalized, justified design, that you will apply to your own project.

THE SYSTEM

A capstone team proposed a Barangay E-Health Record and Referral System, meant to let barangay health workers record patient visits digitally and refer patients to a partner hospital when needed.

At the proposal defense, the panel gave three recommendations. First, the data model had no way to track the status of a referral once it was sent to the hospital. Second, the architecture diagram did not show how the system would work at a barangay station with intermittent internet connectivity. Third, the mockups did not address what a health worker sees when a patient has no prior record.

Resolving Recommendation 1 (Schema)

The team added a new Referral entity with a status attribute that can be Pending, Accepted, Completed, or Cancelled. That entity received three foreign keys: one to the Patient entity, one to the referring Barangay Health Worker, and one to the receiving hospital's Facility entity. The relationship was written out explicitly in the data dictionary: a Patient may have zero or many Referrals, and a Referral must belong to exactly one Patient. This closed the gap directly, and the team logged it in their design review as a resolved recommendation.

Resolving Recommendation 2 (Architecture)

Because barangay stations may lose connectivity, the team revised their architecture from a purely online client-server model to a layered architecture with a local, offline-capable client. That client queues records locally and synchronizes with the central server once a connection returns. Their justification stated plainly that this pattern was chosen over an always-online model because their Functional Requirement 1.4, recording a patient visit without an active internet connection, could not be satisfied without local storage and a synchronization mechanism.

Resolving Recommendation 3 (UI/UX)

The team extended their Figma mockups with an explicit New Patient empty state, shown whenever a health worker searches for a patient and finds no match. A clear call to action lets the worker create a new record instead of reaching a dead end. This reflects the usability heuristic of error prevention and user control, since the system now anticipates a real workflow gap rather than assuming every patient already exists in the database.

The Full Justification

Finally, the team wrote a four-part justification for the revised design as a whole. Functionally, the offline-capable architecture and the new Referral entity satisfy the requirements the panel flagged as unmet. For security, patient data synced from the barangay client is encrypted in transit, and health worker accounts are authenticated before any local queue can be created. For scalability, the schema's foreign keys are indexed, and the design is stated to target the barangay's actual patient volume of roughly 3,000 households. For problem alignment, every one of

these changes traces back to the original problem statement: barangay health workers lacking a reliable way to record and refer patients, rather than adding complexity unrelated to that problem.

Notice the pattern in every step above: a specific panel recommendation, a specific design change, and a specific, named justification. That is exactly the pattern you will use in the activities that follow.

Learning Activities

Complete Activities 1 through 3 in order. Items are numbered continuously through the rest of this module, including the Final Assessment, so keep track of your item numbers as you go.

Activity 1: Knowledge and Comprehension Check

For Items 1 to 5, match each term in Column A with its correct definition in Column B. Write the letter of the matching definition on the blank before each number. Each letter may be used only once.

Column A: Term	Column B: Definition (unordered)
_____ 1. Layered (n-tier) architecture	A. A normalization standard requiring no repeating groups, no partial dependency on part of a composite key, and no transitive dependency between non-key attributes.
_____ 2. Crow's Foot notation	B. An architectural pattern that separates presentation, business logic, and data into distinct layers.
_____ 3. Context Diagram (Level 0)	C. A single deployable application with clearly separated internal modules, often the middle ground between a layered design and true microservices.
_____ 4. Third Normal Form (3NF)	D. A notation that expresses the cardinality of a relationship directly on the connecting line between two entities.
_____ 5. Modular monolith	E. A diagram that represents the entire system as a single process, showing only its boundary and external entities.

For Items 6 to 8, write TRUE or FALSE on the blank before each statement.

6. A capstone team should default to a microservices architecture because it demonstrates the most advanced technical skill. _____

7. A foreign key belongs on the many side of a one-to-many relationship. _____

8. An interface's happy-path screens are sufficient for a validated UI/UX mockup, since error and empty states can be added later during construction. _____

Activity 2: Guided Application Worksheet

Items 9 to 11 use the short scenario below. Items 12 to 14 ask you to apply the same thinking to your own capstone project.

SCENARIO

A proposal panel tells a team: "Your data model has no way to track whether a submitted request was approved, denied, or is still pending."

9. Which schema-refinement checklist item does this recommendation most directly point to, and what new schema element would resolve it?

10. State the cardinality and participation between a new Request entity and an existing Applicant entity, using the sentence form modeled in the worked example (for instance: an X may have zero or many Ys, and a Y must belong to exactly one X).

11. Name two of the four architectural patterns covered in the concept review, and give one capstone-appropriate reason a team might choose each one.

Pattern A:

Pattern B:

12. From your own Capstone 1 panel's recommendations, write down one recommendation that concerns your system's architecture, database schema, or data flow diagrams.

13. Which of the six schema-refinement checklist items (entity completeness, attribute definition, primary and foreign keys, cardinality and participation, normalization, or referential integrity and cascade rules) best addresses the recommendation you wrote in Item 12?

14. Describe, in your own words, the specific schema or architecture change your team will make in response to that recommendation.

Activity 3: Analysis and Synthesis Task

15. A team's barangay-based system was proposed as always-online client-server, but the panel noted that the barangay hall loses internet access for several hours at a time. Analyze this situation: what functionality and usability risks does the original design create for the system's actual end users?

16. Using the four-part justification framework (functionality, security, scalability, and problem alignment), write a short justification for one major design decision in your own capstone project. Address all four dimensions.

17. Identify one usability heuristic from the concept review that your own system's current UI/UX design does not yet fully satisfy, and describe the specific change you will make to satisfy it.

Final Assessment

This assessment continues the item numbering from the Learning Activities above. It has 30 items worth a total of 100 points, in three parts: Part I is objective (Items 18 to 32, 2 points each), Part II is application (Items 33 to 42, 4 points each), and Part III is analysis and essay (Items 43 to 47, 6 points each). Follow your instructor's time limit and instructions for open-book or closed-book conditions. Write your answers directly on this module unless your instructor tells you otherwise.

Name:	Section / Team:	Date:
--------------	------------------------	--------------

Part I. Objective (Items 18 to 32, 2 points each)

For Items 18 to 27, write the letter of the best answer on the blank before each number. For Items 28 to 32, write the single term or short phrase being described.

18. Which architectural pattern separates a system into presentation, business logic, and data layers? _____

- A. Client-server
- B. Layered (n-tier)
- C. Microservices
- D. Model-View-Controller

19. A team proposes true microservices for a system with three users and one functional module. Based on the concept review, what is the most likely panel reaction? _____

- A. Approval, since microservices are always the most modern choice
- B. A request to justify the choice, since microservices are rarely necessary at capstone scale
- C. No comment, since architecture choice does not require justification
- D. A requirement to switch to a distributed database

20. Which of the following is NOT one of the six schema-refinement checklist items? _____

- A. Entity completeness
- B. Attribute definition
- C. Version control branching strategy
- D. Referential integrity and cascade rules

21. A password field in a schema should be documented as _____

- A. A plain-text string field
- B. A field to be hashed before storage
- C. An optional field
- D. A foreign key

22. Which normal form specifically eliminates transitive dependency of a non-key attribute on another non-key attribute? _____

- A. First Normal Form (1NF)
- B. Second Normal Form (2NF)
- C. Third Normal Form (3NF)
- D. Boyce-Codd Normal Form (BCNF)

23. In Crow's Foot notation, cardinality is expressed _____

- A. In a separate table
- B. Directly on the line connecting two entities
- C. Only in the data dictionary
- D. Only through color coding

24. Which DFD level represents the entire system as a single process, showing only its boundary and external entities? _____

- A. Level 2
- B. Level 1
- C. Context Diagram (Level 0)
- D. Sequence diagram

25. A sequence diagram is most useful for documenting _____

- A. Data storage requirements
- B. The step-by-step interaction between the client, the API layer, and the database for one transaction
- C. Color and typography standards
- D. Entity relationships

26. Which usability heuristic is being applied when a system shows a loading indicator while a report is generating?

- A. Consistency and standards
- B. Visibility of system status
- C. Error prevention
- D. Accessibility

27. Which of the four design-justification dimensions is addressed by the statement, "Passwords are hashed using bcrypt before storage"? _____

- A. Functionality
- B. Security
- C. Scalability
- D. Problem alignment

- 28.** The tool named in the concept review for defining a schema through a lightweight text-based syntax that instantly renders a diagram: _____
- 29.** The single deployable application with clearly separated internal modules, often the middle ground between a layered design and true microservices: _____
- 30.** The type of key placed on the many side of a one-to-many relationship: _____
- 31.** The stage of UI/UX refinement where error states, empty states, and confirmation dialogs must be designed, not just the happy path: _____
- 32.** The four dimensions used to justify a design decision in this module: _____, _____, _____, and _____.

Part II. Application (Items 33 to 42, 4 points each)

Items 33 to 38 use the scenario below. Items 39 to 42 ask you to apply the same thinking to your own capstone project.

SCENARIO

A capstone team is building a Cooperative Loan Management System for a multipurpose cooperative. At their Capstone 1 defense, the panel gave three recommendations: (1) the schema does not show how a loan's repayment status is tracked over time; (2) the architecture does not specify what happens if the loan officer's device loses connection while encoding a new loan application; (3) the mockups do not show what a loan officer sees when a member has multiple active loans.

33. Propose a new entity that would resolve Recommendation 1. Give it at least three attributes and complete the table below.

Attribute	Data Type	Constraint

34. Write the cardinality and participation statement, in the sentence form used in this module, between your new entity from Item 33 and the existing Member entity.

35. Which architectural revision from the worked model would most directly resolve Recommendation 2? Explain your answer in two to three sentences.

36. Describe, step by step, the component sequence (client, then which components, in what order) that results from your answer to Item 35.

37. Propose a UI/UX refinement that resolves Recommendation 3, and name the specific usability heuristic it satisfies.

38. State one normalization check (1NF, 2NF, or 3NF) that the new entity from Item 33 should be tested against, and explain in one sentence what that check would verify for this entity.

39. List the primary key and at least two foreign keys for one entity in your own system's finalized ERD.

Key Type	Field Name	References

40. Identify the architectural pattern your team has finalized for your own system, and list the major components shown in your architecture diagram.

41. Identify one Level 1 DFD subprocess in your own system, and the data store or stores it reads from or writes to.

42. Identify one screen in your own system's UI/UX mockups that required an error-state or empty-state addition during refinement, and describe that addition.

Part III. Analysis and Essay (Items 43 to 47, 6 points each)

Answer each item in complete sentences. Suggested lengths are given as a guide, not a strict limit.

43. A classmate argues that writing a four-part justification is unnecessary busywork as long as the system works when demonstrated. Analyze this argument using the concepts from this module, and explain what risk their position creates at final defense. (Suggested length: 5 to 8 sentences)

44. Compare and contrast a layered (n-tier) architecture and a modular monolith architecture in terms of when each is the more defensible choice for a capstone-scale system. (Suggested length: 5 to 8 sentences)

45. Analyze the relationship between the Design Review Checklist and the traceability practice discussed in this module. Why would a design reviewer specifically check that DFD data stores map to ERD entities, and that ERD entities map back to the panel's original recommendations? (Suggested length: 5 to 8 sentences)

46. Write a complete four-part justification (functionality, security, scalability, and problem alignment) for the single most significant design change your team made to your system after your Capstone 1 defense. (Suggested length: one short paragraph per dimension)

47. Reflecting on this module as a whole, identify the one artifact (architecture, schema, DFD, or UI/UX) that required the most revision in your own project, and explain, with specific reference to at least one panel recommendation, why that artifact needed the most work. (Suggested length: 8 to 10 sentences)

Answer Sheet, Part I

Use this sheet to record your final answers for Items 18 to 32. This does not replace your written work above; it is provided so your instructor can check Part I quickly.

Item No.	Answer	Item No.	Answer
18		26	
19		27	
20		28	
21		29	
22		30	
23		31	
24		32	
25			

Score Summary (Instructor Use)

Your instructor will complete this table when checking your Final Assessment.

Part	Items	Points Possible	Score
Part I, Objective	18 to 32 (15 items)	30	
Part II, Application	33 to 42 (10 items)	40	
Part III, Analysis and Essay	43 to 47 (5 items)	30	
Total	18 to 47 (30 items)	100	

Final Group Requirement: Finalizing and Presenting Your System Design Document

This module closes with one requirement that your whole group completes together, separate from the individually written Learning Activities and Final Assessment above. It puts this module's first learning outcome directly into practice, and it is graded separately from the 100-point Final Assessment; it does not add to that total.

YOUR FINAL REQUIREMENT

As your group's final requirement for this module, finalize your system's architecture, database schema, and process/data flow diagrams, based on the recommendations your panel gave you at your Capstone 1 proposal defense.

This finalized output becomes part of the complete System Design Document described in this module's Performance Standard, together with your refined UI/UX mockups. Use the schema-refinement checklist, the DFD leveling guidance, and the worked model in this module as your working reference while you finalize each artifact.

Presentation Requirement

Your group will present this finalized output to the class within 10 to 15 minutes during your next scheduled meeting. Every group member should be prepared to speak for part of the presentation and to answer questions from your instructor afterward.

Suggested Presentation Outline

Use the outline below as a starting structure. Adjust the timing to fit your own content, but stay within the 10 to 15 minute limit.

- **1. Introduction and problem recap, about 1 minute.** State your system's name, the problem it addresses, and its intended end users.
- **2. Panel recommendations recap, about 1 to 2 minutes.** State each recommendation your panel gave at your Capstone 1 proposal defense.
- **3. Finalized architecture, about 2 minutes.** Show your architecture diagram, name the pattern you chose, and explain how it resolves the relevant recommendation.
- **4. Finalized database schema, about 2 minutes.** Show your ERD, and highlight the entities, attributes, or relationships that changed in response to a recommendation.
- **5. Finalized process and data flow diagrams, about 2 minutes.** Show your DFD, and walk through one subprocess in detail.
- **6. Design justification, about 2 to 3 minutes.** Present a four-part justification, functionality, security, scalability, and problem alignment, for your most significant design change.

- **7. Questions and answers, about 2 to 3 minutes.** Be ready for follow-up questions on any artifact your group presented.

When you describe the tools you used, refer to the type of tool, for example a diagramming tool, a database modeling tool, or a UI/UX design tool, rather than a specific product name.

Group Identification

Complete this before your presentation.

Group Number	
Project / System Title	
Member 1 (Name and Role)	
Member 2 (Name and Role)	
Member 3 (Name and Role)	
Member 4 (Name and Role)	
Member 5 (Name and Role)	

Group Presentation Rubric (Preview)

Your instructor will grade this presentation using the four criteria below. Each is worth 25 points, for a total of 100 points.

Criterion	Points	What This Assesses
Design Completeness and Technical Accuracy	25	Whether your finalized architecture, database schema, and process/data flow diagrams are complete, technically sound, and consistent with each other.
Incorporation of Panel Recommendations	25	Whether every recommendation your panel gave at your Capstone 1 proposal defense is clearly and specifically addressed.
Design Justification	25	Whether your four-part justification, functionality, security, scalability, and problem alignment, is specific and accurate, not generic.
Presentation Delivery and Team Participation	25	Whether the presentation is organized, stays within the 10 to 15 minute limit, and involves every group member.